جامعة أم القرى
UMM AL-QURA UNIVERSITY

Cinema Project

Course Title:
Database (DS2201)

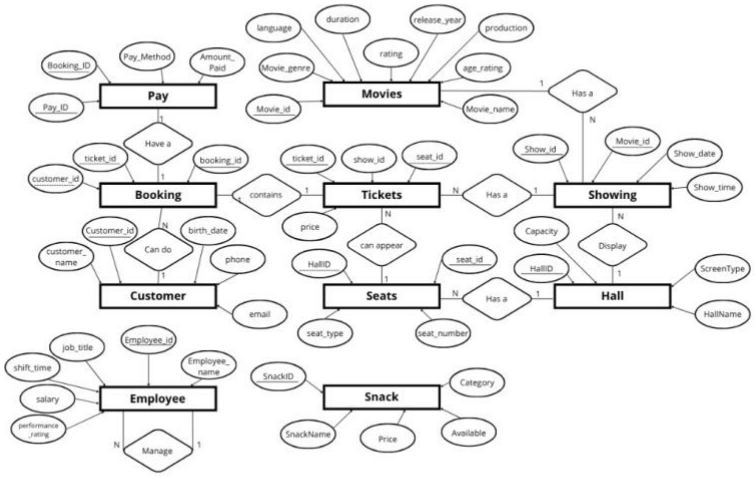
Instructor:
Dr.Ohoud Alzahrani

Team Members:
Marya Alghamdi 446002296

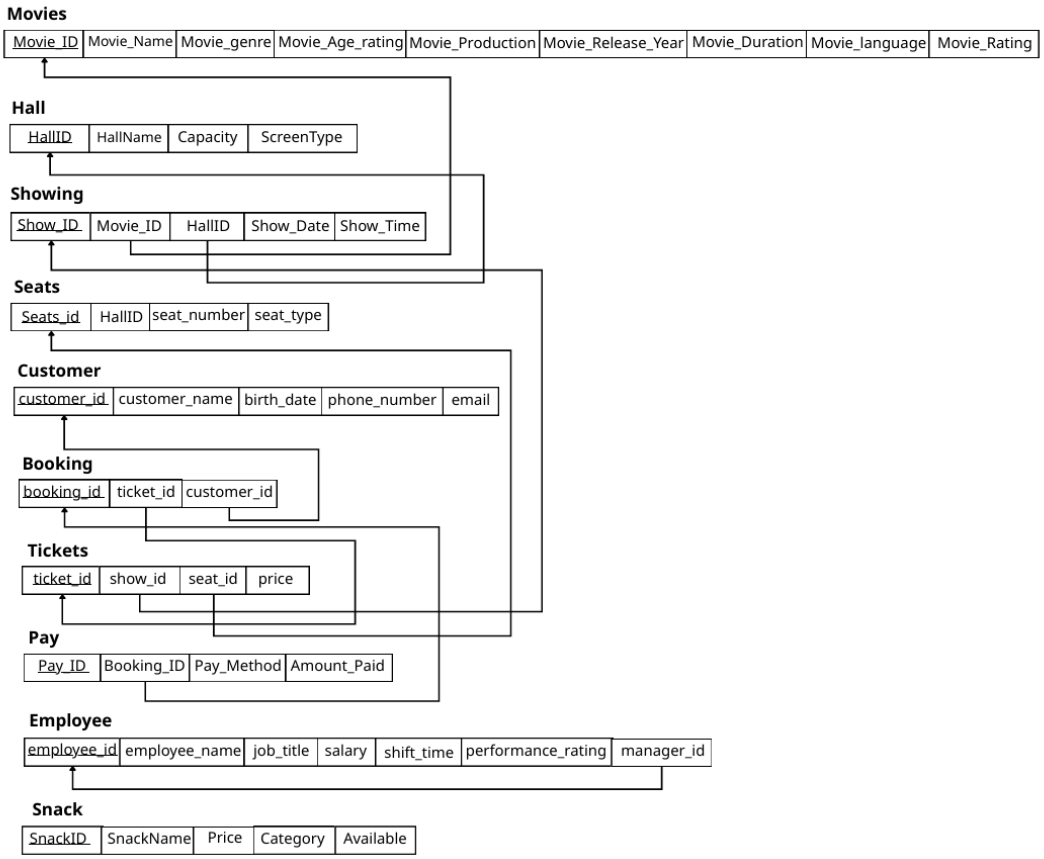
1. Organization Description

This project designs a database system for managing a cinema. It organizes data related to movies, halls, customers, employees, bookings, and snacks. The system supports movie scheduling through the ShowTime table, links customers to bookings and orders, and follows normalization principles to ensure data accuracy and reduce redundancy.

2.Relationship Diagram (EERD):



2. Database Schema :



4. Normalization

Database normalization serves the purpose of structuring data to achieve both reduced duplication and enhanced data accuracy. The project implements normalization methods to transform a cinema management system database until it meets the standards of Third Normal Form (3NF).

Initial Tables

The database consists of the following main tables:

- Movies
- Showing
- Hall
- Seats
- Tickets
- Booking
- Pay
- Customer
- Snack
- Employee

Functional Dependencies

The main functional dependencies identified in the system are:

- Movie_ID → Movie_Name, Genre, Age_Rating, Production, Release_Year, Duration, Language, Rating
- Show_ID → Movie_ID, HallID, Show_Date, Show_Time
- HallID → HallName, Capacity, ScreenType
- Seat_ID → HallID, Seat_Number, Seat_Type
- Ticket_ID → Show_ID, Seat_ID, Price
- Booking_ID → Ticket_ID, Customer_ID
- Pay_ID → Booking_ID, Pay_Method, Amount_Paid
- Customer_ID → Name, Birth_Date, Phone, Email
- Snack_ID → SnackName, Price, Category, Availability
- Employee_ID → Name, Job_Title, Salary, Shift_Time, Performance_Rating

First Normal Form (1NF)

All tables satisfy the requirements of 1NF because:

- All attributes contain atomic (indivisible) values
- There are no repeating groups or multi-valued attributes

Second Normal Form (2NF)

All tables are in 2NF because:

- Each table has a single primary key
- All non-key attributes fully depend on the primary key
- There are no partial dependencies

Third Normal Form (3NF)

Most tables already satisfy 3NF. However, one important issue was identified:

Transitive Dependency in Movies Table

- Movie_ID → Movie_Production
- Movie_Production → Movie_Language

This creates a transitive dependency.

Normalization Solution

To resolve the transitive dependency, the Movies table was decomposed into two tables:

Movies Table

Movies(

Movie_ID PRIMARY KEY,

Movie_Name,

Movie_Genre,

Movie_Age_Rating,

Movie_Production,

Movie_Release_Year,

Movie_Duration,

Movie_Rating

)

Production Table

Production(

Movie_Production PRIMARY KEY,

Movie_Language

)

Optional Optimization

Employee Table Improvement

A possible functional dependency exists:

- **Job_Title → Salary**

To improve normalization:

- **Job Table**

Job(

Job_Title PRIMARY KEY,

Salary

)

- **Employee Table**

Employee(

Employee_ID PRIMARY KEY,

Employee_Name,

Job_Title,

Shift_Time,

Performance_Rating

)

Ticket Pricing Optimization (Optional)

Ticket price may depend on seat type:

- Seat_Type → Price

Optional improvement:

SeatType(

Seat_Type PRIMARY KEY,

Price

)

9. Final Normalized Schema (3NF)

The database after normalization consists of:

- Movies
- Production
- Showing
- Hall
- Seats
- Tickets
- Booking
- Pay
- Customer
- Snack
- Employee
- Job (optional)
- SeatType (optional)

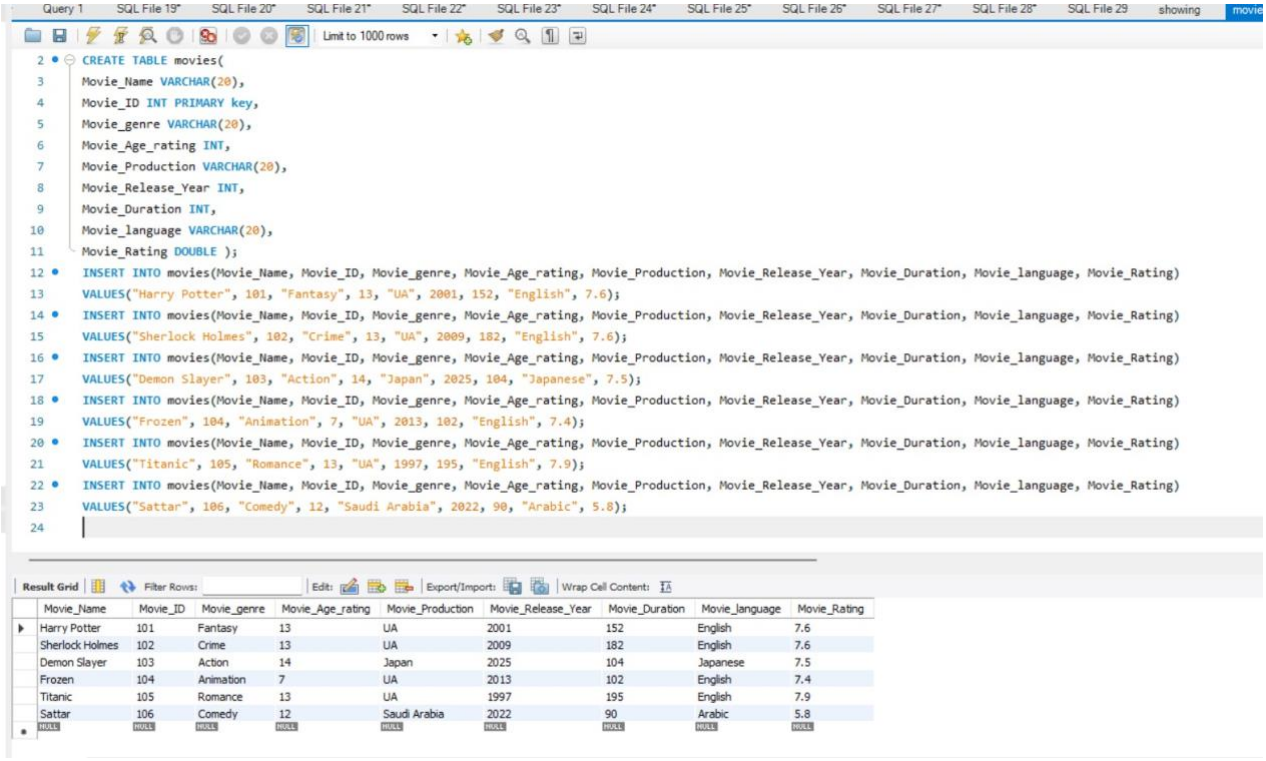
Conclusion

The cinema database has been successfully normalized up to Third Normal Form (3NF). All tables satisfy the requirements of 1NF and 2NF, and transitive dependencies were resolved. The final design reduces redundancy, improves data consistency, and enhances maintainability.

5. MySQL Implementation

5.1 Database and Table Creation

Movies Table :



```
CREATE TABLE movies(
Movie_Name VARCHAR(20),
Movie_ID INT PRIMARY key,
Movie_genre VARCHAR(20),
Movie_Age_rating INT,
Movie_Production VARCHAR(20),
Movie_Release_Year INT,
Movie_Duration INT,
Movie_language VARCHAR(20),
Movie_Rating DOUBLE );
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Harry Potter", 101, "Fantasy", 13, "UA", 2001, 152, "English", 7.6);
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Sherlock Holmes", 102, "Crime", 13, "UA", 2009, 182, "English", 7.6);
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Demon Slayer", 103, "Action", 14, "Japan", 2025, 104, "Japanese", 7.5);
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Frozen", 104, "Animation", 7, "UA", 2013, 102, "English", 7.4);
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Titanic", 105, "Romance", 13, "UA", 1997, 195, "English", 7.9);
INSERT INTO movies(Movie_Name, Movie_ID, Movie_genre, Movie_Age_rating, Movie_Production, Movie_Release_Year, Movie_Duration, Movie_language, Movie_Rating)
VALUES("Sattar", 106, "Comedy", 12, "Saudi Arabia", 2022, 90, "Arabic", 5.8);
```

Movie_Name	Movie_ID	Movie_genre	Movie_Age_rating	Movie_Production	Movie_Release_Year	Movie_Duration	Movie_language	Movie_Rating
Harry Potter	101	Fantasy	13	UA	2001	152	English	7.6
Sherlock Holmes	102	Crime	13	UA	2009	182	English	7.6
Demon Slayer	103	Action	14	Japan	2025	104	Japanese	7.5
Frozen	104	Animation	7	UA	2013	102	English	7.4
Titanic	105	Romance	13	UA	1997	195	English	7.9
Sattar	106	Comedy	12	Saudi Arabia	2022	90	Arabic	5.8
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hall Table :

```sql
CREATE TABLE Hall (
    HallID INT PRIMARY KEY,
    HallName VARCHAR(30),
    Capacity INT,
    ScreenType VARCHAR(20)
);
INSERT INTO Hall VALUES (401,'Hall A',4,'IMAX');
INSERT INTO Hall VALUES (402,'Hall B',4,'3D');
INSERT INTO Hall VALUES (403,'Hall C',4,'Normal');
```

Result Grid | Filter Rows: | Edit: | Export/Import:

HallID	HallName	Capacity	ScreenType
401	Hall A	4	IMAX
402	Hall B	4	3D
403	Hall C	4	Normal
NULL	NULL	NULL	NULL

Showing Table :

movies | Query 1 | SQL File 19* | SQL File 20* | SQL File 21* | SQL File 22* | SQL File 23* | SQL File 24* | SQL File 25* | SQL File 26*

Limit to 1000 rows

```
1   USE cinema;
2   CREATE TABLE Showing(
3   Show_ID INT PRIMARY key,
4   Movie_ID INT, FOREIGN KEY(Movie_ID) REFERENCES movies(Movie_ID),
5   HallID INT, FOREIGN KEY(HallID) REFERENCES Hall(HallID),
6   Show_Date DATE,
7   Show_Time TIME
8   );
9   INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(201, 101, 401, '2026-05-01', '14:00:00');
10  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(202, 101, 402, '2026-05-04', '16:00:00');
11  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(203, 102, 401, '2026-05-02', '17:00:00');
12  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(204, 102, 403, '2026-05-04', '19:00:00');
13  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(205, 103, 401, '2026-05-03', '14:00:00');
14  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(206, 103, 402, '2026-05-01', '17:15:00');
15  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(207, 104, 403, '2026-05-03', '16:30:00');
16  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(208, 104, 402, '2026-05-02', '19:20:00');
17  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(209, 105, 401, '2026-05-04', '21:30:00');
18  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(210, 105, 402, '2026-05-03', '18:30:00');
19  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(211, 106, 403, '2026-05-01', '20:00:00');
20  INSERT INTO Showing(Show_ID, Movie_ID, HallID, Show_Date, Show_Time)VALUES(212, 106, 403, '2026-05-02', '21:30:00');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Show_ID	Movie_ID	HallID	Show_Date	Show_Time
201	101	401	2026-05-01	14:00:00
202	101	402	2026-05-04	16:00:00
203	102	401	2026-05-02	17:00:00
204	102	403	2026-05-04	19:00:00
205	103	401	2026-05-03	14:00:00
206	103	402	2026-05-01	17:15:00
207	104	403	2026-05-03	16:30:00
208	104	402	2026-05-02	19:20:00
209	105	401	2026-05-04	21:30:00
210	105	402	2026-05-03	18:30:00
211	106	403	2026-05-01	20:00:00
212	106	403	2026-05-02	21:30:00
NULL	NULL	NULL	NULL	NULL

showing 1 ×

Output

Seats Table:

```sql
CREATE TABLE Seats (
    seat_id INT PRIMARY KEY,
    HallID INT, FOREIGN KEY(HallID) REFERENCES Hall(HallID),
    seat_number VARCHAR(10),
    seat_type VARCHAR(20)
);
INSERT INTO Seats (seat_id, HallID, seat_number, seat_type) VALUES
(701, 401, 'A1', 'Regular'),
(702, 401, 'A2', 'Regular'),
(703, 401, 'A3', 'VIP'),
(704, 401, 'A4', 'VIP'),
(705, 402, 'B1', 'Regular'),
(706, 402, 'B2', 'Regular'),
(707, 402, 'B3', 'VIP'),
(708, 402, 'B4', 'VIP'),
(709, 403, 'C1', 'Regular'),
(710, 403, 'C2', 'Regular'),
(711, 403, 'C3', 'VIP'),
(712, 403, 'C4', 'VIP');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

seat_id	HallID	seat_number	seat_type
701	401	A1	Regular
702	401	A2	Regular
703	401	A3	VIP
704	401	A4	VIP
705	402	B1	Regular
706	402	B2	Regular
707	402	B3	VIP
708	402	B4	VIP
709	403	C1	Regular
710	403	C2	Regular
711	403	C3	VIP
712	403	C4	VIP
NULL	NULL	NULL	NULL

seats 1 ×

Tickets Table :

```sql
CREATE TABLE Tickets (
    ticket_id INT PRIMARY KEY,
    show_id INT, FOREIGN KEY (show_id) REFERENCES Showing(Show_ID),
    seat_id INT, FOREIGN KEY (seat_id) REFERENCES Seats(seat_id),
    price DOUBLE
    );
INSERT INTO Tickets (ticket_id, show_id, seat_id, price) VALUES
(801, 201, 701, 50),
(802, 201, 702, 50),
(803, 201, 703, 80),
(804, 201, 704, 80),

(805, 202, 705, 50),
(806, 202, 706, 50),
(807, 202, 707, 80),
(808, 202, 708, 80),

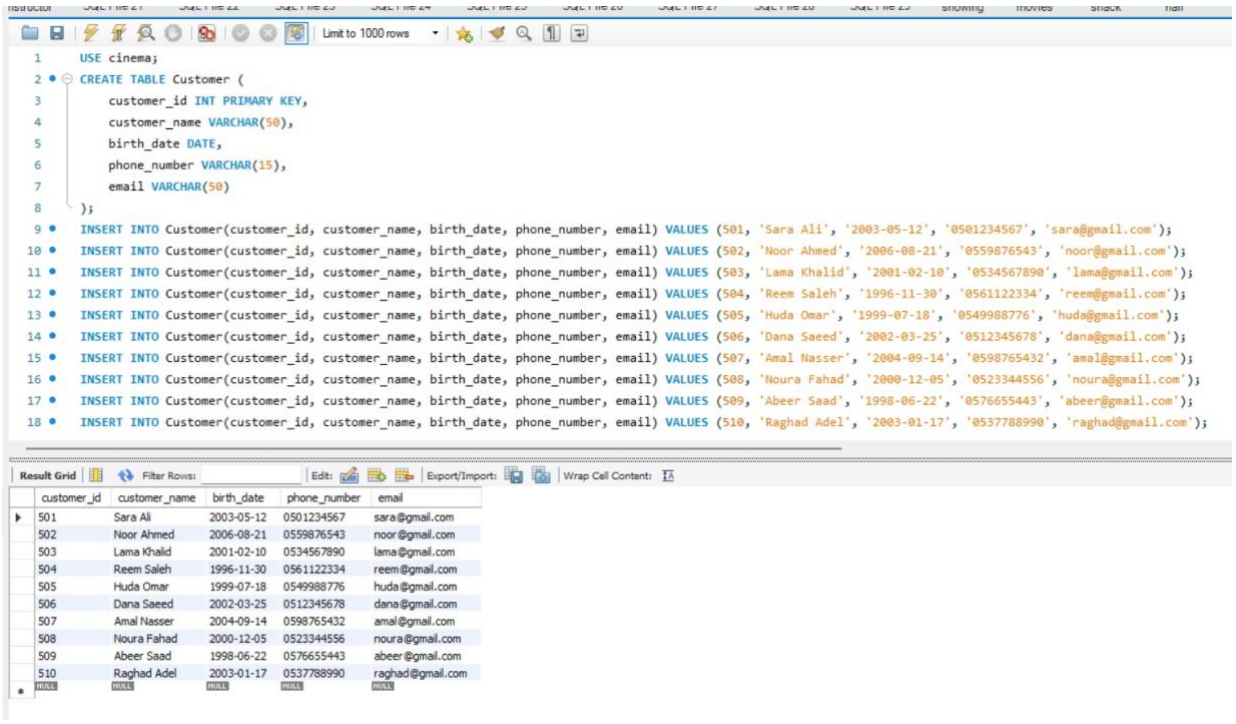
(809, 203, 701, 50),
(810, 203, 702, 50),
(811, 203, 703, 80),
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

ticket_id	show_id	seat_id	price
801	201	701	50
802	201	702	50
803	201	703	80
804	201	704	80
805	202	705	50
806	202	706	50
807	202	707	80
808	202	708	80
809	203	701	50
810	203	702	50
811	203	703	80
812	203	704	80

tickets 1 ×

Customer Table :



```sql
USE cinema;
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    birth_date DATE,
    phone_number VARCHAR(15),
    email VARCHAR(50)
);
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (501, 'Sara Ali', '2003-05-12', '0501234567', 'sara@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (502, 'Noor Ahmed', '2006-08-21', '0559876543', 'noor@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (503, 'Lama Khalid', '2001-02-10', '0534567890', 'lama@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (504, 'Reem Saleh', '1996-11-30', '0561122334', 'reem@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (505, 'Huda Omar', '1999-07-18', '0549988776', 'huda@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (506, 'Dana Saeed', '2002-03-25', '0512345678', 'dana@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (507, 'Amal Nasser', '2004-09-14', '0598765432', 'amal@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (508, 'Noura Fahad', '2000-12-05', '0523344556', 'noura@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (509, 'Abeer Saad', '1998-06-22', '0576655443', 'abeer@gmail.com');
INSERT INTO Customer(customer_id, customer_name, birth_date, phone_number, email) VALUES (510, 'Raghad Adel', '2003-01-17', '0537788990', 'raghad@gmail.com');
```

Result Grid

customer_id	customer_name	birth_date	phone_number	email
501	Sara Ali	2003-05-12	0501234567	sara@gmail.com
502	Noor Ahmed	2006-08-21	0559876543	noor@gmail.com
503	Lama Khalid	2001-02-10	0534567890	lama@gmail.com
504	Reem Saleh	1996-11-30	0561122334	reem@gmail.com
505	Huda Omar	1999-07-18	0549988776	huda@gmail.com
506	Dana Saeed	2002-03-25	0512345678	dana@gmail.com
507	Amal Nasser	2004-09-14	0598765432	amal@gmail.com
508	Noura Fahad	2000-12-05	0523344556	noura@gmail.com
509	Abeer Saad	1998-06-22	0576655443	abeer@gmail.com
510	Raghad Adel	2003-01-17	0537788990	raghad@gmail.com
NULL	NULL	NULL	NULL	NULL

Booking Table :

```sql
USE cinema;
CREATE TABLE Booking (
    booking_id INT PRIMARY KEY,
    ticket_id INT, FOREIGN KEY(ticket_id) REFERENCES Tickets(ticket_id),
    customer_id INT, FOREIGN KEY(customer_id) REFERENCES Customer(customer_id)
);
INSERT INTO Booking (booking_id, ticket_id, customer_id) VALUES
(001, 801, 501),
(002, 803, 502),
(003, 806, 503),
(004, 811, 504),
(005, 815, 505),
(006, 818, 506),
(007, 823, 507),
(008, 827, 508),
(009, 832, 509),
(010, 835, 510),
(011, 842, 501),
(012, 847, 503);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

booking_id	ticket_id	customer_id
1	801	501
2	803	502
3	806	503
4	811	504
5	815	505
6	818	506
7	823	507
8	827	508
9	832	509
10	835	510
11	842	501
12	847	503
NULL	NULL	NULL

Delete selected rows

booking 1

Pay Table :

```sql
USE cinema;
CREATE TABLE Pay (
    Pay_ID INT PRIMARY KEY,
    Booking_ID INT, FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),
    Pay_Method VARCHAR(30),
    Amount_Paid DOUBLE
);
INSERT INTO Pay (Pay_ID, Booking_ID, Pay_Method, Amount_Paid) VALUES
(1001, 001, 'Apple Pay', 50),
(1002, 002, 'Credit Card', 80),
(1003, 003, 'Cash', 50),
(1004, 004, 'Apple Pay', 80),
(1005, 005, 'Credit Card', 80),
(1006, 006, 'Cash', 50),
(1007, 007, 'Apple Pay', 80),
(1008, 008, 'Credit Card', 80),
(1009, 009, 'Cash', 80),
(1010, 010, 'Apple Pay', 80),
(1011, 011, 'Credit Card', 50),
(1012, 012, 'Cash', 80);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

Pay_ID	Booking_ID	Pay_Method	Amount_Paid
1001	1	Apple Pay	50
1002	2	Credit Card	80
1003	3	Cash	50
1004	4	Apple Pay	80
1005	5	Credit Card	80
1006	6	Cash	50
1007	7	Apple Pay	80
1008	8	Credit Card	80
1009	9	Cash	80
1010	10	Apple Pay	80
1011	11	Credit Card	50
1012	12	Cash	80
NULL	NULL	NULL	NULL

pay 1 ×

Employee Table :

instructor SQL File 22* SQL File 23* SQL File 24* SQL File 25* SQL File 26* SQL File 27* SQL File 28* SQL File 29

Limit to 1000 rows

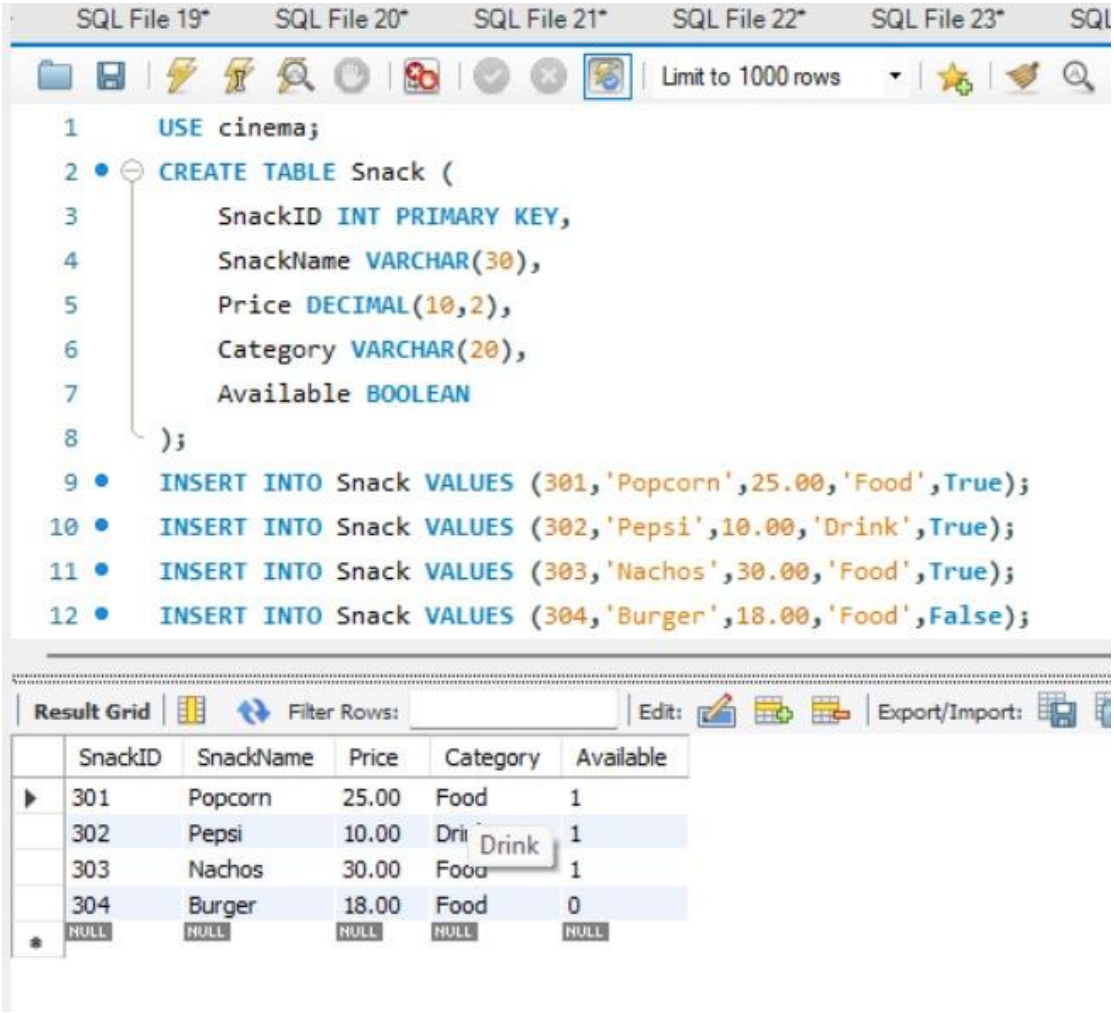
```
1   USE cinema;
2   CREATE TABLE Employee (
3       employee_id INT PRIMARY KEY,
4       employee_name VARCHAR(50),
5       job_title VARCHAR(30),
6       salary INT,
7       shift_time VARCHAR(20),
8       performance_rating INT
9   );
10  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
11  (601, 'Ahmed Saleh', 'Ticket Seller', 5000, '1pm-6pm', 4);
12  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
13  (606, 'Fahad Ali', 'Ticket Seller', 5000, '7pm-12am', 5);
14  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
15  (602, 'Mona Ali', 'Cashier', 4800, '7pm-12am', 4);
16  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
17  (603, 'Khalid Noor', 'Manager', 8000, '1pm-6pm', 5);
18  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
19  (604, 'Rania Omar', 'Cleaner', 3000, '7pm-12am', 3);
20  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
21  (607, 'Salem Hassan', 'Cleaner', 3000, '1pm-6pm', 4);
22  INSERT INTO Employee(employee_id, employee_name, job_title, salary, shift_time, performance_rating) VALUES
23  (605, 'Faisal Sami', 'Security', 4500, '7pm-12am', 4);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	employee_name	job_title	salary	shift_time	performance_rating
601	Ahmed Saleh	Ticket Seller	5000	1pm-6pm	4
602	Mona Ali	Cashier	4800	7pm-12am	4
603	Khalid Noor	Manager	8000	1pm-6pm	5
604	Rania Omar	Cleaner	3000	7pm-12am	3
605	Faisal Sami	Security	4500	7pm-12am	4
606	Fahad Ali	Ticket Seller	5000	7pm-12am	5
607	Salem Hassan	Cleaner	3000	1pm-6pm	4
NULL	NULL	NULL	NULL	NULL	NULL

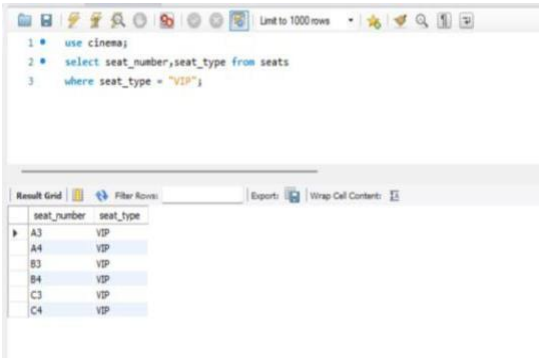
employee 1 ×

Snack Table :


SQL File 19*
SQL File 20*
SQL File 21*
SQL File 22*
SQL File 23*
Limit to 1000 rows
USE cinema;
CREATE TABLE Snack (
SnackID INT PRIMARY KEY,
SnackName VARCHAR(30),
Price DECIMAL(10,2),
Category VARCHAR(20),
Available BOOLEAN
);
INSERT INTO Snack VALUES (301,'Popcorn',25.00,'Food',True);
INSERT INTO Snack VALUES (302,'Pepsi',10.00,'Drink',True);
INSERT INTO Snack VALUES (303,'Nachos',30.00,'Food',True);
INSERT INTO Snack VALUES (304,'Burger',18.00,'Food',False);
Result Grid
Filter Rows:
Edit:
Export/Import:
SnackID SnackName Price Category Available
301 Popcorn 25.00 Food 1
302 Pepsi 10.00 Drink 1
303 Nachos 30.00 Food 1
304 Burger 18.00 Food 0
NULL NULL NULL NULL NULL

6. SQL Queries

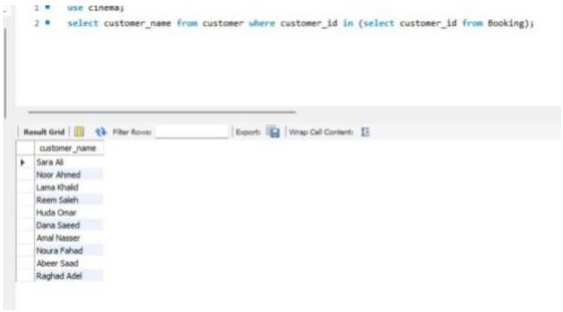
Query 1 :



```
use cinema;
select seat_number,seat_type from seats
where seat_type = "VIP";
```

seat_number	seat_type
A3	VIP
A4	VIP
B3	VIP
B4	VIP
C3	VIP
C4	VIP

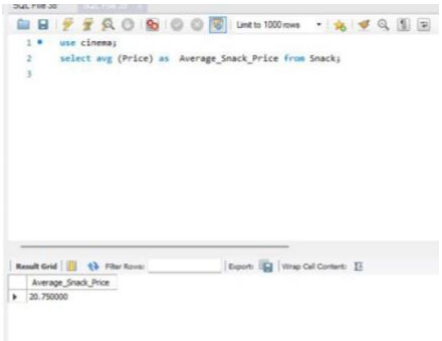
Query 2 :



```
use cinema;
select customer_name from customer where customer_id in (select customer_id from Booking);
```

customer_name
Sara Ali
Noor Ahmed
Lama Khalid
Reem Saleh
Huda Omar
Dana Saeed
Amal Nasser
Noura Fahad
Abeer Saad
Raghad Adel

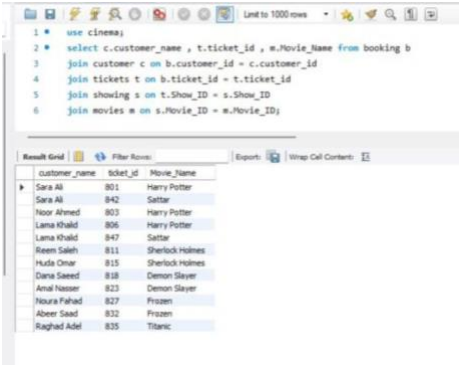
Query 3 :



```
use cinema;
select avg (Price) as  Average_Snack_Price from Snack;
```

Average_Snack_Price
20.750000

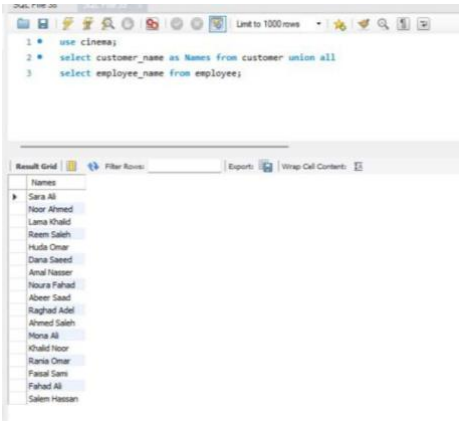
Query 4 :



```
use cinema;
select c.customer_name , t.ticket_id , m.Movie_Name from booking b
join customer c on b.customer_id = c.customer_id
join tickets t on b.ticket_id = t.ticket_id
join showing s on t.Show_ID = s.Show_ID
join movies m on s.Movie_ID = m.Movie_ID;
```

customer_name	ticket_id	Movie_Name
Sara Ali	801	Harry Potter
Sara Ali	842	Sattar
Noor Ahmed	803	Harry Potter
Lama Khalid	806	Harry Potter
Lama Khalid	847	Sattar
Reem Saleh	811	Sherlock Holmes
Huda Omar	815	Sherlock Holmes
Dana Saeed	818	Demon Slayer
Amal Nasser	823	Demon Slayer
Noura Fahad	827	Frozen
Abeer Saad	832	Frozen
Raghad Adel	835	Titanic

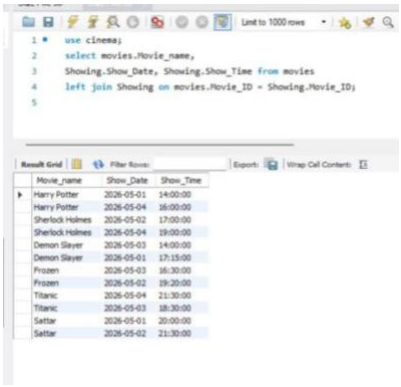
Query 5 :



```
use cinema;
select customer_name as Names from customer union all
select employee_name from employee;
```

Names
Sara Ali
Noor Ahmed
Lama Khalid
Reem Saleh
Huda Omar
Dana Saeed
Amal Nasser
Noura Fahad
Abeer Saad
Raghad Adel
Ahmed Saleh
Mona Ali
Khalid Noor
Rania Omar
Faisal Sami
Fahad Ali
Salem Hassan

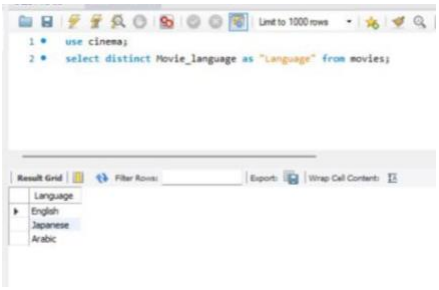
Query 6 :



```
use cinema;
select movies.Movie_name,
Showing.Show_Date, Showing.Show_Time from movies
left join Showing on movies.Movie_ID = Showing.Movie_ID;
```

Movie_name	Show_Date	Show_Time
Harry Potter	2026-05-01	14:00:00
Harry Potter	2026-05-04	16:00:00
Sherlock Holmes	2026-05-02	17:00:00
Sherlock Holmes	2026-05-04	19:00:00
Demon Slayer	2026-05-03	14:00:00
Demon Slayer	2026-05-01	17:15:00
Frozen	2026-05-03	16:30:00
Frozen	2026-05-02	19:20:00
Titanic	2026-05-04	21:30:00
Titanic	2026-05-03	18:30:00
Sattar	2026-05-01	20:00:00
Sattar	2026-05-02	21:30:00

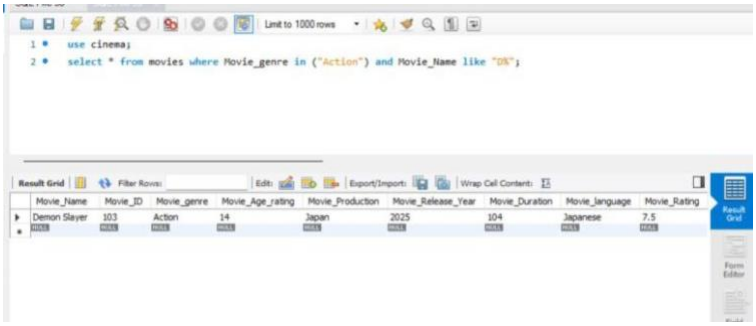
Query 7 :



```
use cinema;
select distinct Movie_language as "Language" from movies;
```

Language
English
Japanese
Arabic

Query 8 :



```
use cinema;
select * from movies where Movie_genre in ("Action") and Movie_Name like "D%";
```

Movie_Name	Movie_ID	Movie_genre	Movie_Age_rating	Movie_Production	Movie_Release_Year	Movie_Duration	Movie_language	Movie_Rating
Demon Slayer	103	Action	14	Japan	2025	104	Japanese	7.5

Query 9

```
use cinema;
select job_title, count(*) from employee
group by job_title
having count(*)>1
order by count(*) desc ;
```

job_title	count(*)
Ticket Seller	2
Cleaner	2

Query 10:

```
use cinema;
select employee_name , salary from employee
order by salary asc;
```

employee_name	salary
Rania Omar	3000
Salem Hassan	3000
Faisal Sami	4500
Mona Ali	4800
Ahmed Saleh	5000
Fahad Ali	5000
Khalid Noor	8000